# 1. Asymptotic Analysis

Topic: asymptotic-analysis

Asymptotic analysis describes how an algorithm's running time grows as the input size n becomes large. We ignore constants and lower-order terms and focus on the dominant term.

## The three notations

- **Big-O (upper bound):** f(n) = O(g(n)) if there exist constants c > 0 and n0 such that f(n) <= c * g(n) for all n >= n0.
- **Big-Omega (lower bound):** f(n) = Omega(g(n)) if f(n) >= c * g(n) for all n >= n0.
- **Big-Theta (tight bound):** f(n) = Theta(g(n)) if f(n) is both O(g(n)) and Omega(g(n)).

## Example

f(n) = 3n^2 + 5n + 2. For n >= 1, f(n) <= 3n^2 + 5n^2 + 2n^2 = 10n^2, so f(n) = O(n^2) with c = 10, n0 = 1. Since f(n) >= 3n^2, it is also Omega(n^2). Therefore f(n) = Theta(n^2).

## Common growth rates (slowest to fastest)

1 < log n < n < n log n < n^2 < n^3 < 2^n < n!

## Best, worst and average case

The worst case is the maximum number of operations over all inputs of size n; the best case is the minimum; the average case is the expected number over a probability distribution of inputs. In SC2001 we usually measure cost by counting **key comparisons** rather than wall-clock time, because comparisons dominate the work in sorting and searching.

# 2. Recurrences and the Master Theorem

Topic: recurrences

Divide-and-conquer algorithms have running times described by recurrences. The Master Theorem solves recurrences of the form:

```
T(n) = a T(n/b) + f(n),   a >= 1, b > 1
```

Compare f(n) with n^(log_b a), which measures the total work done at the leaves of the recursion tree.

- **Case 1:** if f(n) = O(n^(log_b a - e)) for some e > 0, the leaves dominate: T(n) = Theta(n^(log_b a)).
- **Case 2:** if f(n) = Theta(n^(log_b a)), every level does equal work: T(n) = Theta(n^(log_b a) * log n).
- **Case 3:** if f(n) = Omega(n^(log_b a + e)) for some e > 0 AND the regularity condition a f(n/b) <= c f(n) holds for some c < 1, the root dominates: T(n) = Theta(f(n)).

## Worked examples

- Mergesort: T(n) = 2T(n/2) + n. Here n^(log_2 2) = n = f(n), so Case 2 gives Theta(n log n).
- Binary search: T(n) = T(n/2) + 1. Here n^(log_2 1) = 1 = f(n), so Case 2 gives Theta(log n).
- T(n) = 8T(n/2) + n^2. Here n^(log_2 8) = n^3, which grows faster than n^2, so Case 1 gives Theta(n^3).

## When the Master Theorem does not apply

If the subproblems are not of equal size, or f(n) falls between cases (for example f(n) = n log n with a = b = 2), use the iteration (substitution) method: expand the recurrence repeatedly, find the pattern, and sum the work across all levels.

# 3. Insertion Sort and Mergesort

Topic: sorting-insertion-merge

## Insertion sort

Insertion sort builds a sorted prefix one element at a time, inserting each new element into its correct position by comparing it with elements to its left.

- Worst case (reverse-sorted input): n(n-1)/2 comparisons, Theta(n^2).
- Best case (already sorted input): n - 1 comparisons, Theta(n).
- In-place and stable. Very fast for small or nearly-sorted arrays.

## Mergesort

Mergesort is divide and conquer: split the array into two halves, recursively sort each half, then merge the two sorted halves.

- Merging two sorted lists of sizes m and n takes at most m + n - 1 comparisons.
- Time: Theta(n log n) in the best, worst and average case.
- Stable, but needs Theta(n) extra memory for merging (not in-place).

## Hybrid sort (insertion sort + mergesort)

Mergesort's recursion overhead is wasteful on tiny subarrays. A hybrid algorithm recurses like mergesort but switches to insertion sort once a subarray has size <= S. Its time complexity is Theta(nS + n log(n/S)): insertion sort costs about S^2 per small subarray across n/S subarrays, and merging costs n per level across log(n/S) levels. The best threshold S is found empirically by counting key comparisons for different S values.

# 4. Quicksort

Topic: sorting-quicksort

Quicksort picks a **pivot**, partitions the array so that elements smaller than the pivot are on its left and larger elements are on its right, then recursively sorts both sides. Unlike mergesort, the work happens before the recursive calls.

## Complexity

- **Best and average case:** Theta(n log n). On average quicksort makes about 1.39 n log2 n comparisons.
- **Worst case:** Theta(n^2). This happens when the pivot is always the smallest or largest element, for example choosing the first element as pivot on an already-sorted array. The partitions become sizes 0 and n - 1.
- Partitioning n elements takes n - 1 comparisons.

## Properties

- In-place (only O(log n) stack space on average).
- Not stable: equal elements may change relative order.
- Usually faster than mergesort in practice due to good cache behaviour and small constants.

## Avoiding the worst case

**Randomized pivot:** choose the pivot uniformly at random, making the worst case extremely unlikely for any input. **Median-of-three:** use the median of the first, middle and last elements as pivot, which handles sorted and reverse-sorted input well.

# 5. Heaps and Heapsort

Topic: sorting-heapsort

A **binary heap** is a complete binary tree stored in an array. In a **max-heap**, every parent is greater than or equal to its children, so the maximum is always at the root.

## Array representation (1-indexed)

```
parent(i) = floor(i/2)    left(i) = 2i    right(i) = 2i + 1
```

## Core operations

- **FixHeap (sift-down):** restores the heap property at a node by swapping it with its larger child until it is in place. Cost O(log n), the height of the tree.
- **Heap construction (bottom-up):** call FixHeap on every non-leaf node from the last one up to the root. Total cost is O(n), not O(n log n), because most nodes are near the bottom and sift down only a short distance.
- **DeleteMax:** remove the root, move the last element to the root, then FixHeap. Cost O(log n).

## Heapsort

Build a max-heap in O(n), then repeatedly swap the root with the last element of the heap, shrink the heap by one and FixHeap. Each of the n deletions costs O(log n), giving Theta(n log n) in the best, worst and average case.

- In-place, but not stable.
- Guaranteed n log n with no extra memory, unlike quicksort (bad worst case) and mergesort (extra memory).

Heaps also implement **priority queues**, which are used in Dijkstra's algorithm.

# 6. Graph Representation and Traversal (BFS, DFS)

Topic: graph-traversal

## Representations

- **Adjacency matrix:** a V x V table. Theta(V^2) space; checking whether edge (u, v) exists is O(1). Good for dense graphs.
- **Adjacency list:** each vertex stores a list of its neighbours. Theta(V + E) space. Good for sparse graphs.

## Breadth-first search (BFS)

BFS explores the graph level by level from a source vertex using a **queue**. It visits all vertices at distance 1, then distance 2, and so on. BFS finds shortest paths (fewest edges) in an **unweighted** graph.

## Depth-first search (DFS)

DFS goes as deep as possible along one path before backtracking, using a **stack** or recursion. DFS is used for cycle detection, finding connected components, and topological sorting of a directed acyclic graph (DAG).

## Complexity

- With an adjacency list, both BFS and DFS run in Theta(V + E): each vertex is visited once and each edge is examined a constant number of times.
- With an adjacency matrix, both run in Theta(V^2), because finding a vertex's neighbours requires scanning its whole row.

# 7. Dijkstra's Shortest Path Algorithm

Topic: dijkstra

Dijkstra's algorithm solves the **single-source shortest path** problem on a weighted graph with **non-negative** edge weights. It is a greedy algorithm.

## Algorithm

```
d[source] = 0; d[v] = infinity for all other v
while unvisited vertices remain:
    u = unvisited vertex with smallest d[u]
    mark u as visited (d[u] is now final)
    for each edge (u, v) with weight w:
        if d[u] + w < d[v]:
            d[v] = d[u] + w      # relaxation
            pi[v] = u            # predecessor
```

## Time complexity depends on the data structures

- **Adjacency matrix + array for d:** finding the minimum is O(V) per step, done V times, and scanning neighbours is O(V) per vertex. Total O(V^2). Good for dense graphs.
- **Adjacency list + min-heap priority queue:** extracting the minimum costs O(log V) and each relaxation (decrease-key) costs O(log V). Total O((V + E) log V). Good for sparse graphs.

## Limitations

Dijkstra fails with **negative edge weights**: once a vertex is finalized, the algorithm assumes no shorter path exists, but a negative edge discovered later could create one. Use the Bellman-Ford algorithm for graphs with negative weights, in O(VE) time.

# 8. Dynamic Programming

Topic: dynamic-programming

Dynamic programming (DP) solves problems by combining solutions to subproblems, storing each subproblem's answer so it is computed only once. It applies when a problem has:

- **Optimal substructure:** an optimal solution is built from optimal solutions to subproblems.
- **Overlapping subproblems:** the same subproblems are solved many times by naive recursion.

## Two styles

- **Top-down (memoization):** write the natural recursion, but cache each result in a table.
- **Bottom-up (tabulation):** fill a table iteratively from the smallest subproblems upward.

Example: naive recursive Fibonacci takes exponential time because fib(n-2) is recomputed many times. DP computes each fib(i) once, giving O(n).

## 0/1 Knapsack

Given n items with weights w_j and profits p_j, and capacity C, choose items (each at most once) to maximize total profit. Let P(c, j) be the best profit using the first j items with capacity c:

```
P(c, j) = P(c, j-1)                                 if w_j > c
P(c, j) = max( P(c, j-1), P(c - w_j, j-1) + p_j )   otherwise
P(c, 0) = 0,  P(0, j) = 0
```

Filling the table takes O(nC) time. This is **pseudo-polynomial**: it is polynomial in the numeric value C, but exponential in the number of bits needed to write C.

## Unbounded knapsack

If each item can be used any number of times: P(c) = max over items j with w_j <= c of ( P(c - w_j) + p_j ), with P(0) = 0.